

Indkapsling



Program:

- Access modifiers
- Indkapsling
- Getter og Setter metoder
- Application Programming Interface (API)
- Øvelser
- Opsummering

A young woman with long, wavy blonde hair is smiling and looking off to the side. She is wearing a blue and white horizontally striped sweater. The background is a blurred outdoor setting with a brick building and some greenery.

Access Modifiers

So far, har vi brugt **public** til alt

- klasser:

```
public class BankAccount { // public class
    public double balance; // public field

    public BankAccount(int accountNumber) { // public constructor
        this.accountNumber = accountNumber;
        this.balance = 0.0;
    }

    public void deposit(double amount) { // public method
        this.balance += amount;
    }
}
```

public er en access modifier (dan: adgangsmodifikator)

Java har fire access modifiers:

- `public` betyder at alle kan se og bruge det
- `private` betyder at kun klassen selv kan se og bruge det
- `protected` betyder at kun klassen selv og dens subklasser kan se og bruge det
- (default) betyder at kun klassen selv og klasser *i samme pakke* kan se og bruge det

Måske har du set **private** før - ved et uheld

DEMO - Extract method i IntelliJ



A group of four people are walking across a bridge, each pushing a bicycle. From left to right: a man in a grey t-shirt and khaki shorts holding a coffee cup; a woman in a yellow top and blue jeans; a man in a white t-shirt and grey trousers; and a woman in a grey jacket and blue jeans. They are all smiling and looking towards the right. The bridge has a metal railing and a paved path. In the background, there are city buildings, including a church with a dome, and a clear blue sky with some clouds. The text "Hvorfor indkapsling?" is overlaid in a white box across the middle of the image.

Hvorfor indkapsling?

Kan i huske bankkonto eksemplet?

```
public class BankAccount {  
    public int accountNumber;  
    public double balance;  
}
```

Vi kan oprette en bankkonto og ændre dens saldo direkte

```
BankAccount account = new BankAccount(337898);  
account.balance = -1000.0; // uh oh, negativ saldo!
```

Så lavede vi en **deposit** og **withdraw** metode for at undgå det

```
public class BankAccount {  
    public int accountNumber;  
    public double balance;  
  
    public void deposit(double amount) {  
        if (amount > 0) {  
            this.balance += amount;  
        }  
    }  
    public void withdraw(double amount) {  
        if (amount > 0 && amount <= this.balance) {  
            this.balance -= amount;  
        }  
    }  
}
```

De metoder sikrer at saldoen ikke kan blive negativ

```
BankAccount account = new BankAccount(337898);  
account.deposit(1000.0);  
account.withdraw(500.0); // OK  
account.withdraw(600.0); // ikke OK, saldoen er kun 500.0  
  
account.deposit(-200.0); // ikke OK, kan ikke indsætte negative beløb
```

MEN - vi kan stadig ændre saldoen direkte

```
BankAccount account = new BankAccount(337898);  
account.balance = -1000.0; // uh oh, negativ saldo!
```

Det er her indkapsling kommer ind i billedet

I al fremtid: Gør felter **private**

Før:

```
public class BankAccount {  
    public int accountNumber;  
    public double balance;  
}
```

Nu:

```
public class BankAccount {  
    private int accountNumber;  
    private double balance;  
}
```

Ellers ligner resten af klassen sig selv

```
public class BankAccount {  
    private int accountNumber;  
    private double balance;  
  
    public BankAccount(int accountNumber) {  
        this.accountNumber = accountNumber;  
        this.balance = 0.0;  
    }  
  
    public void deposit(double amount) {  
        if (amount > 0) {  
            this.balance += amount;  
        }  
    }  
  
    public void withdraw(double amount) {  
        if (amount > 0 && amount <= this.balance) {  
            this.balance -= amount;  
        }  
    }  
}
```

Nu kan vi ikke længere ændre **balance** direkte

```
BankAccount account = new BankAccount(337898);  
account.balance = -1000.0; // fejl! balance er utilgængelig
```

men vi kan stadig ændre den via **deposit** og **withdraw**

```
account.deposit(500.0); // OK  
account.withdraw(200.0); // OK
```

A photograph of two men sitting at a desk in a modern office, looking at two computer monitors displaying code. The man on the left has a beard and is wearing a dark jacket over a plaid shirt. The man on the right is wearing a plaid shirt. The office has large windows in the background showing a city street with buildings. A white text box is overlaid on the image, containing the title.

Getter og Setter metoder

Hvordan får vi adgang til `balance` og `accountNumber` nu?

```
BankAccount account = new BankAccount(337898);  
System.out.println(account.balance); // fejl! balance er u
```

Løsningen er at lave getter og setter metoder

DEMO - getters og setters

```
public class BankAccount {
    private int accountNumber;
    private double balance;

    // ... konstruktør, deposit, withdraw ...

    public int getAccountNumber() {
        return this.accountNumber;
    }

    public void setAccountNumber(int accountNumber) {
        this.accountNumber = accountNumber;
    }

    public double getBalance() {
        return this.balance;
    }

    public void setBalance(double balance) {
        if (balance >= 0) {
            this.balance = balance;
        }
    }
}
```

Nu kan vi få adgang til **balance** og **accountNumber** via getters

```
BankAccount account = new BankAccount(337898);  
System.out.println(account.getBalance()); // 0.0  
account.deposit(500.0);  
System.out.println(account.getBalance()); // 500.0  
System.out.println(account.getAccountNumber()); // 337898
```

Men vi kan også ændre dem via setters

```
BankAccount account = new BankAccount(337898);  
  
account.setBalance(1000.0); // OK  
account.setAccountNumber(123456); // OK
```

Argh...

Det har vi set før

Spørgsmål: Hvordan gør vi accountNumber skrivebeskyttet (read-only)?

Spørgsmål: Skal vi gøre balance til en konstant (final)?

Trick: Vi gør **setBalance** metoden **private**

```
public class BankAccount {  
    private int accountNumber;  
    private double balance;  
  
    public BankAccount(int accountNumber) {  
        this.accountNumber = accountNumber;  
        this.balance = 0.0;  
    }  
  
    public int getAccountNumber() {  
        return this.accountNumber;  
    }  
  
    public double getBalance() {  
        return this.balance;  
    }  
  
    private void setBalance(double balance) {  
        this.balance = balance;  
    }  
  
    // ... deposit og withdraw ...  
}
```

Nu kan vi kun ændre **balance** via **deposit** og **withdraw**

```
BankAccount account = new BankAccount(337898);  
account.setBalance(1000.0); // fejl! setBalance er private  
account.deposit(500.0); // OK  
account.withdraw(200.0); // OK
```

Hvad skal vi så med en `setBalance` metode?

```
public void deposit(double amount) {  
    if (amount > 0) {  
        setBalance(this.balance + amount);  
    }  
}  
  
public void withdraw(double amount) {  
    if (amount > 0 && amount <= this.balance) {  
        setBalance(this.balance - amount);  
    }  
}  
  
private void setBalance(double balance) {  
    this.balance = balance;  
}
```

kan vi forenkle `deposit` og `withdraw` ved at flytte valideringen til `setBalance`?

```
public void deposit(double amount) {  
    setBalance(this.balance + amount);  
}  
  
public void withdraw(double amount) {  
    setBalance(this.balance - amount);  
}  
  
private void setBalance(double balance) {  
    if (balance >= 0) {  
        this.balance = balance;  
    }  
}
```

Ja, det kan vi. Snedigt - ik?

Følgende er ikke eksamensrelevant - men blot til refleksion

API

KEA
Guldbergsgade 29N

Spørgsmål: Har nogen af jer hørt om **API** før?

API står for **A**pplication **P**rogramming **I**nterface

Et API er en samling af klasser, metoder og felter, **som udstilles for andre udviklere**, så de kan bruge dem i deres programmer

`public` klasser, metoder og felter er API'et,
som udviklere der bruger klassen kan se og bruge

`private` klasser, metoder og felter er
implementation detaljer, som kun klassen selv bruger

I vores **BankAccount** eksempel er flg. en del af vores API:

- klassen `BankAccount` (da den er `public`)
- konstruktøren `BankAccount(int accountNumber)` (da den er `public`)
- metoderne
 - `getAccountNumber()`,
 - `getBalance()`,
 - `deposit(double amount)` og
 - `withdraw(double amount) ...` da de er `public`

Spørgsmål: Kan vi ikke bare gøre alt **public**
og så lade brugeren om at bruge det *rigtigt*?

Nej!

Nej, fordi:

- Det er svært at forstå/bruge et API korrekt, hvis alt er `public`
- Det er svært at ændre et API, hvis alt er `public`
- Vi kan utilsigtet bryde kode, der bruger vores API, hvis alt er `public`
- Vi er nødt til at lægge alt kode i den metode, der bruger det, hvis alt er `public`

```
private void setBalance(double balance) {  
    if (balance >= 0) {  
        this.balance = balance;  
    }  
}
```

... hvis `setBalance(...)` var `public`,
hvorfor bruge `deposit(...)` og `withdraw(...)`?

Vi kan ændre reglerne for hvordan **balance** opdateres, *uden* at bryde kode, der bruger vores API, fx

```
private void setBalance(double balance) {  
    if (balance >= 0 && balance <= 1_000_000) { // max saldo 1 million  
        this.balance = balance;  
    }  
}
```

Versionering af API'er

A blurred background image of a person with red hair and freckles, looking over a dark surface. The person's face is partially visible on the right side of the frame, with their eyes looking towards the camera. The background is out of focus, showing indistinct shapes and colors.

Når vi ændrer et API, skal vi overveje om ændringen er **bagud kompatibel** eller ej

- En **bagud kompatibel** ændring betyder at kode, der bruger API'et, stadig virker uden ændringer
- En **ikke bagud kompatibel** ændring betyder at kode, der bruger API'et, skal ændres for at virke med den nye version af API'et

I software udvikling, er det almindeligt at vise om en ny version af et API er bagud kompatibel eller ej ved hjælp af versionsnumre

Semver (semantisk versionering) bruger tre tal:

- version 1.0.0
- version 1.1.0 (bagud kompatibel ændring af version 1.0.0)
- version 1.1.1 (bagud kompatibel **fejlrettelse** af version 1.1.0)
- version 2.0 (*ikke* bagud kompatibel ændring af version 1.1)

- -> 2.0.1 (fejlrettelser, bagud kompatibel)
- -> 2.1.0 (ny feature, bagud kompatibel)
- -> 2.0.0 (breaking change)

Så ja, rigtige udviklere tænker over hvilke dele af deres kode, der skal være **public**
og hvilke der skal være **private**

Nævn tre ting du tager med fra i dag?